AI Intelligence Ingestion Pipeline — Architecture

GraphOne / FrontierAtlas AI Engineer trial | Scale, resilience, and storage design for a 500,000+ record intelligence graph

1. Concurrency architecture

Every network call flows through one **AsyncHttpClient** (aiohttp + a pooled TCPConnector), gated by two asyncio.Semaphore layers: a global cap (MAX_CONCURRENCY) and a per-host cap (PER_HOST_CONCURRENCY), so one slow host can never starve the rest of a batch. A generic **run_bounded()** runner wraps every crawl loop: it fans work out to at most N in-flight coroutines and isolates per-item failures with a try/except around each task, so one malformed page can't abort a 500k-record run. Politeness-constrained sources (arXiv's API terms ask for ~1 request/3s) are the one exception: pagination there is sequential with a fixed delay, but the per-page *post-processing* (GitHub lookup, validation, DB write) still fans out through the same bounded runner, since that work isn't subject to the source's own rate limit.

2. Scale strategy: 500,000+ startups, products, and papers

Nothing in the code path changes between 1,000 and 500,000 records — only configuration and horizontal infrastructure do:

- **Concurrency is a config value.** MAX_CONCURRENCY / PER_HOST_CONCURRENCY are the only knobs AsyncHttpClient reads; raising them, plus running more worker processes/containers each with their own budget, is the entire scale-up story.
- **Pagination is already generator-based** (ArxivExtractor.iter_category, RateLimitedPager) — "fetch more" is a loop bound, not new logic.
- **Partitioning is a checkpoint-namespace split.** CheckpointStore keys state per source; splitting one source's ID space (arXiv categories, YC batches, alphabetic shards of a company list) across N workers, each with its own namespace (or a shared Postgres checkpoint table in production), turns one crawler into a fleet without touching extraction logic.
- **Storage swaps via one env var** (DATABASE_URL) from SQLite to Postgres — the SQLAlchemy models are unchanged, so the schema is proven at 1k before it ever sees production write volume.
- **Orchestration**: in production this becomes N worker pods (Kubernetes Jobs or a Celery/RQ queue) reading partitioned work from a queue, each running the identical run_bounded()-based extractor, writing to the same Postgres primary. A scheduler (Airflow/Dagster) owns retry-the-whole-partition semantics on top of the per-item retries already built in.

3. Handling 413s and 429s across thousands of concurrent extractions

413 (payload too large) — never a blind character-count truncation:

- clean_html() strips boilerplate (script/style/nav/footer/ads by tag and class heuristics) and locates the main content block before anything is sized.
- chunk_text() packs paragraphs into token-budgeted chunks (≈4 chars/token estimate) with a small overlap across chunk boundaries, and always prepends the lead paragraph to every chunk — the highest-density sentence survives even if a fallback provider only ever sees chunk 2.
- If a provider still 413s (its real limit is stricter than assumed), shrink_for_413() halves the token budget and retries that *same* provider up to max_413_retries times before falling through to the next tier — a small-context provider doesn't fail the whole extraction.

429 (rate limited) — exponential backoff with jitter, honoring a Retry-After header when the provider sends one; exhausting max_429_retries on one provider falls through to the next tier rather than continuing to hammer it. The same retry/backoff primitive (src/utils/retry.py) is shared by the HTTP crawler layer and the LLM layer, so 429 handling behaves identically everywhere instead of being reimplemented per call site.

Provider chain: Gemini → Groq → DeepSeek, each an interchangeable adapter behind one interface (LLMProvider.call); LLMOrchestrator.configured_providers filters out any tier with no API key, so the chain degrades gracefully to whichever tiers are actually configured — down to zero, where deterministic keyword-based fallbacks (pricing/role classification) keep the pipeline producing valid output with no LLM key at all.

Failure	Detection	Response	Escalation
429	HTTP 429 / RateLimitError	Backoff + jitter, honor Retry-After	Exhaust retries → next provider tier
413	HTTP 413 / PayloadTooLargeError	Halve token budget, re-chunk, retry	Exhaust 413 retries → next provider tier
Timeout/DNS/TLS	aiohttp exception after retry_async	Return (599, None, {}) — never raises	Isolated by run_bounded; batch continues

4. Freshness tracking and distributed deduplication

Two independent layers, so the same article/job is never processed twice — even across distributed crawler nodes:

- **Checkpoint dedup** (per source ID / URL): CheckpointStore is a SQLite table today (WAL mode + busy_timeout for concurrent writers); at fleet scale this becomes one shared Postgres table (namespace, item_id) with a unique constraint — every worker, regardless of node, checks and marks against the same table, so two nodes racing on the same page can't both emit the same

item. A row is marked *seen* only after its DB insert commits, not before — marking first would permanently skip an item on resume if the insert then failed.

- **Content-hash dedup** (cross-source): a SHA-256 of normalized text (whitespace/case-collapsed) catches the same story syndicated to two outlets, or a job cross-posted to two boards, which URL-based dedup alone would miss.
- **Freshness itself is strict, not assumed**: parse_datetime() normalizes ISO-8601, RFC-822/RSS, epoch, and relative strings ("2 hours ago") to UTC; is_fresh() returns False for anything unparseable — a missing date is never treated as fresh. The one documented heuristic fallback (a source with no timestamp at all) only fires against real prior-crawl checkpoint state, and explicitly refuses to mark anything fresh on that source's first-ever run, so a full backlog can't flood in as false-fresh on day one.

5. Storage strategy

Primary: PostgreSQL. The workload is fundamentally relational — typed records with foreign-key-shaped relationships (a Product belongs to a canonical Startup; a Job belongs to a canonical Company) — and needs real ACID transactions under many concurrent writers, which SQLite's single-writer model doesn't provide at production scale. The demo in this repo runs on SQLite by design (zero setup, identical SQLAlchemy models), and DATABASE_URL is the only thing that changes to point at Postgres — the schema is proven before it ever carries production write volume.

Vector/graph strategy: pgvector, not a separate graph database. Rather than standing up Neo4j for a 1,000-record trial, the extensible design adds a pgvector column to the existing Postgres tables — embed entity names/descriptions once and use pgvector's ANN index for "similar startups"/"similar papers" queries in the *same* database and the *same* transaction as everything else. If multi-hop relationship traversal becomes the dominant query pattern at real scale ("startups founded by alumni of X"), this Postgres data exports cleanly into Neo4j; provisioning that speculatively now would be infrastructure for its own sake, which the assignment explicitly asks not to do.

6. Entity resolution

Deterministic, ordered, and fully audited: Unicode NFKD normalize → lowercase → strip punctuation → collapse whitespace → strip a trailing *legal* suffix (Inc/LLC/Corp/GmbH, iteratively) — deliberately excluding brand words like "Labs"/"Technologies", since auto-stripping those raises false-merge risk. Exact match against a 56-entity seed+alias table, then alias table, then rapidfuzz token_sort_ratio fuzzy match at a confidence threshold (default 97) — below threshold, the input becomes its own new canonical entity rather than being glued onto something unrelated. Every resolution, including "no match, new canonical," is written to an audit log (raw name, canonical name, method, confidence, source URL, timestamp) in both CSV and the database, so the log is a complete trail, not just the interesting cases.

7. Source traceability

Every record carries its originating source.name/source.url end to end from extraction through storage and export — nothing is derived without a citable origin. Research-paper GitHub links are the strictest example: a repo is attached to a paper only when that paper's own abstract/comment field contains the exact URL (is_valid_github_evidence), never inferred from title similarity, and the DB stores that link as null rather than guessing when no such evidence exists. Every rejected record is logged with its reason and source URL (logs/rejected_<run_id>.jsonl) instead of being silently dropped, so "why isn't this record here" is always answerable from the logs.

8. Production scaling & operations

- **Structured logging** (structlog, JSON) throughout, with per-run quality-stats counters (discovered/fetched/validated/rejected/duplicates/429s/413s/freshness-failures) — the same counters that let this document's own numbers be verified rather than estimated.
- **Graceful shutdown**: a cooperative flag checked before each new task starts, wired to SIGINT/SIGTERM where the platform supports it (POSIX; Windows falls back to a top-level KeyboardInterrupt handler) — in-flight work finishes, no new work starts, checkpoints are already durable so nothing is lost.
- **Horizontal scaling path**: containerize the CLI, run N replicas against partitioned checkpoint namespaces and a shared Postgres, front the LLM tier with per-provider concurrency limits (a provider's own rate limit is global across all your workers, not per-worker) enforced via a shared token-bucket (Redis) once beyond a single-process deployment.